

— 1교시 정리

환경과 기본 문법

이 시간에 쓴 코드와 실행 결과

데이터 불러오기

형식

pd.read_csv(파일경로, parse_dates=[열이름, ...])

예제

```
import pandas as pd

df = pd.read_csv('superstore_orders.csv',
                 parse_dates=['Order Date'])

print(df.shape)
```

실행 결과

(10239, 21)

매개변수	의미
parse_dates	날짜로 변환할 열 목록
encoding	파일 인코딩 (기본 utf-8)
sep	구분 기호 (기본 ,)
usecols	읽어 올 열만 지정

엑셀 파일은 pd.read_excel() 을 쓴다.

1.2

크기 · 열 · 자료형

예제

```
print(df.shape)
print(df.columns[:4])
print(df[['Order Date', 'Region',
          'Sales', 'Quantity']].dtypes)
```

`shape` 와 `columns` 뒤에는 괄호를 붙이지 않는다. 함수가 아니라 속성이기 때문이다.

실행 결과

```
(10239, 21)
```

```
Index(['Row ID', 'Order ID',
       'Order Date', 'Ship Date'])
```

```
Order Date    datetime64[ns]
Region         object
Sales          float64
Quantity       int64
dtype: object
```

자료형

표시	자료형	예 · 가능한 연산
int64	정수	3 · 합계, 평균, 크기 비교
float64	실수	16.45 · 합계, 평균, 크기 비교
object	문자열	'West' · 개수 세기, 포함 여부
datetime64	날짜/시각	2023-01-03 · 연·월 추출, 기간 계산
bool	참/거짓	True · 조건 필터

숫자처럼 보여도 자료형이 object 이면 더하기가 이어 붙이기로 동작한다.

1.4

내용 훑어보기

함수	기능
<code>df.head(n)</code>	위에서 n행 (기본 5)
<code>df.tail(n)</code>	아래에서 n행
<code>df.sample(n)</code>	무작위 n행

예제

```
df[['Order ID', 'Region',  
    'Sales', 'Quantity']].head(3)
```

실행 결과				
	Order ID	Region	Sales	Quantity
0	US-2023-103800	Central	16.448	2
1	US-2023-112326	Central	3.540	2
2	US-2023-112326	Central	11.784	3

위쪽만 보면 뒤쪽의 이상을 놓치기 쉬우므로 세 가지를 함께 확인한다.

1.5

요약 정보

예제

```
df[['Sales', 'Quantity', 'Profit']].describe()
```

실행 결과

	Sales	Quantity	Profit
count	10239.00	10239.00	10239.00
mean	239.35	3.78	28.61
std	1206.96	2.24	232.30
min	0.00	-7.00	-6599.98
50%	53.72	3.00	8.69
max	104835.00	14.00	8399.98

평균 239.35 와 중앙값 53.72 의 차이, Quantity 의 최솟값 -7 처럼 확인이 필요한 값이 여기서 드러난다. 해석은 4교시에서 다룬다.

1.6

결측치 확인

형식

```
df.isna().sum()
```

예제

```
na = df.isna().sum()  
print(na[na > 0])
```

`isna()` 와 `isnull()` 은 같은 기능이다. 이름만 두 가지다.

실행 결과

```
Ship Date      321  
Postal Code     60  
dtype: int64
```

결측치를 발견하면 곧바로 지우지 않는다. 왜 비었는지를 먼저 확인한다. 처리 방법은 2교시에서 다룬다.

1.7

열 선택

형식

```
df['열이름']          → Series    (1차원)  
df[['열1', '열2']]    → DataFrame (2차원)
```

예제

```
print(type(df['Sales']))  
print(type(df[['Sales']]))
```

실행 결과

```
<class 'pandas...Series'>  
<class 'pandas...DataFrame'>
```

열 이름은 대소문자와 공백까지 정확히 일치해야 한다. 틀리면 `KeyError` 가 발생한다.

값의 빈도와 종류

함수	기능
<code>s.value_counts()</code>	값 종류별 개수
<code>s.nunique()</code>	값이 몇 종류인지
<code>s.unique()</code>	값의 목록

예제

```
print(df['Region'].value_counts())
```

실행 결과	
Region	
West	3268
East	3002
Central	2343
South	1626
Name: count, dtype: int64	

1.9

행 수와 주문 수

예제

```
print(len(df))  
print(df['Order ID'].nunique())
```

실행 결과

10239

5111

한 주문에 품목이 여럿이면 품목마다 한 행이 생겨 **Order ID** 가 반복된다.

따라서 **행 수(10,239)** 와 **주문 수(5,111)** 가 다르다. 분석 전에 한 행이 무엇인지 반드시 확인한다.

자주 만나는 오류

오류	의미	흔한 원인
KeyError	그런 열이 없다	열 이름 오타 · 공백 · 대소문자
NameError	그런 이름이 없다	위쪽 셀을 실행하지 않음
AttributeError	그런 기능이 없다	함수 이름 오타 · 자료형이 다름
FileNotFoundError	파일을 찾을 수 없다	경로 오류

오류 메시지는 맨 아랫줄에 원인이 적혀 있다. 그 줄을 그대로 검색하거나 AI 에게 붙여 넣으면 대부분 해결된다.

한 장 정리

<code>pd.read_csv()</code> · <code>pd.read_excel()</code>	파일 읽기
<code>df.shape</code> · <code>df.columns</code> · <code>df.dtypes</code>	크기 · 열 이름 · 자료형
<code>df.head()</code> · <code>df.tail()</code> · <code>df.sample()</code>	내용 확인
<code>df.info()</code> · <code>df.describe()</code>	요약 정보 · 기초 통계
<code>df.isna().sum()</code>	열별 결측치 개수
<code>df['열']</code> · <code>df[['열1', '열2']]</code>	Series · DataFrame
<code>s.value_counts()</code> · <code>s.nunique()</code>	값 빈도 · 값 종류 수